

EXPERIMENT - 20

1. Problem Statement

Prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs. Then develop, compile, and execute a menu-driven modular C project that calculates the geometrical measurements and physical properties of selected solids. Test the program using the prepared test cases.

Note: Use only fundamental data types, functions with call by value, selection statements, and loops. Do not use recursion, arrays, pointer variables, pointer parameters, structures, unions, enumerations, typedef, or functions from math.h. Assume that all input values and calculated results are within the ranges supported by their respective data types.

Organization of the project

Module	Functions / Responsibility
main.c	Displays the main menu, obtains common inputs, calls the required functions, and displays results
geometry.c	Calculates surface area, volume, base area, and slant height
physics.c	Obtains the solid type and required dimensions for physics operations and performs the corresponding calculations
math_utils.c	Calculates square root iteratively using the Newton-Raphson method
solid.h	Contains the header guard, symbolic constants, and external function declarations

2. Algorithm

main()

Step 1: START

Step 2: Repeat Steps 3 to 5 using a do-while loop, until the choice is 10.

Step 3: Display the main menu and read choice:

1. Surface area 2. Volume 3. Base area 4. Mass 5. Weight
6. Gravitational potential energy 7. Pressure on the base
8. Buoyant force 9. Buoyancy behaviour 10. Exit

Step 4: Using switch (choice), do the following:

Choice 1 and 2: Display the submenu of solids and read the solid. If it is not from 1 to 4, display the error message and read it again. Read the dimensions needed by that solid, and after every dimension, if it is not greater than 0, display the error message and read it again. Then call sphere_area / cylinder_area / cone_area / cuboid_area for choice 1, or the matching volume function for choice 2.

Choice 3: Read the solid in the same way. If the solid is SPHERE, display the message that the base area is not defined for a sphere and return to the main menu. Otherwise call circular_base_area for a cylinder or a cone, and cuboid_base_area for a cuboid.

Choice 4: Read and validate the density, then call calculate_mass(density).

Choice 5: Call calculate_weight().

Choice 6: Read the elevation. If it is negative, display the error message and read it again. Then call potential_energy(elevation).

Choice 7: Read and validate the density, then call calculate_pressure(density).

Choice 8: Read and validate the fluid density, then call buoyant_force(fluid_density).

Choice 9: Read and validate the density of the solid and the density of the fluid, then call determine_buoyancy(solid_density, fluid_density). If the value returned is SINKS, FLOATS or SUSPENDED, display the matching message.

Choice 10: Display the exit message.

default: Display "Invalid choice".

Step 5: Display the result with its SI unit. Area in square metres, volume in cubic metres, mass in kilograms, weight and buoyant force in newtons, pressure in pascals and potential energy in joules.

Step 6: STOP

Module: geometry.c

Function	Calculate and return
sphere_area(radius)	$4 * PI * radius * radius$
sphere_volume(radius)	$(4.0 / 3.0) * PI * radius * radius * radius$
cylinder_area(radius, height)	$2 * PI * radius * (radius + height)$
cylinder_volume(radius, height)	$PI * radius * radius * height$
cone_area(radius, height)	Step 1: find slant_height as the square root of $radius * radius + height * height$, by calling calculate_square_root() Step 2: return $PI * radius * (radius + slant_height)$
cone_volume(radius, height)	$(1.0 / 3.0) * PI * radius * radius * height$

Function	Calculate and return
cuboid_area(length, breadth, height)	$2 * (\text{length} * \text{breadth} + \text{breadth} * \text{height} + \text{length} * \text{height})$
cuboid_volume(length, breadth, height)	$\text{length} * \text{breadth} * \text{height}$
circular_base_area(radius)	$\text{PI} * \text{radius} * \text{radius}$ (used for a cylinder and a cone)
cuboid_base_area(length, breadth)	$\text{length} * \text{breadth}$

The ideal geometrical base area of a sphere is not defined by the given dimensions, so it is not calculated.

Module: physics.c

calculate_mass(density)

- Step 1:** Read the solid type. If it is not from 1 to 4, display the error message and read it again.
- Step 2:** Read the dimensions needed by that solid, and validate every dimension immediately after it is read.
- Step 3:** Find the volume by calling sphere_volume(), cylinder_volume(), cone_volume() or cuboid_volume().
- Step 4:** Calculate and return mass = density * volume, in kilograms.

calculate_weight()

- Step 1:** Read and validate the density.
- Step 2:** Obtain the mass by calling calculate_mass(density).
- Step 3:** Calculate and return weight = mass * GRAVITY, in newtons.

potential_energy(elevation)

- Step 1:** Read and validate the density.
- Step 2:** Obtain the mass by calling calculate_mass(density).
- Step 3:** Calculate and return potential energy = mass * GRAVITY * elevation, in joules.

calculate_pressure(density)

- Step 1:** Read the solid type, allowing only a cylinder, a cone or a cuboid. If it is not from 2 to 4, display the error message and read it again.
- Step 2:** Read and validate the dimensions needed by that solid.
- Step 3:** Obtain the contact area from circular_base_area() for a cylinder or a cone, and from cuboid_base_area() for a cuboid.
- Step 4:** Obtain the volume from cylinder_volume(), cone_volume() or cuboid_volume().
- Step 5:** Calculate and return pressure = (density * volume * GRAVITY) / contact_area, in pascals.
- Note:** The pressure of a sphere is not calculated, because its ideal geometrical contact area is not defined by the given dimensions.

buoyant_force(fluid_density)

- Step 1:** Read and validate the solid type from 1 to 4.
- Step 2:** Read and validate the dimensions needed by that solid.
- Step 3:** Obtain the volume from sphere_volume(), cylinder_volume(), cone_volume() or cuboid_volume().
- Step 4:** Calculate and return buoyant force = fluid_density * volume * GRAVITY, in newtons.

determine_buoyancy(solid_density, fluid_density)

- Step 1:** If solid_density is greater than fluid_density, return SINKS.
- Step 2:** If solid_density is less than fluid_density, return FLOATS.

Step 3: If the two densities are equal, return SUSPENDED.

Step 4: The corresponding message is displayed in main().

Module: math_utils.c

calculate_square_root(number)

Step 1: If the number is negative, return -1, because its real square root cannot be calculated.

Step 2: If the number is zero, return 0.

Step 3: Set old_value = number.

Step 4: Using a do-while loop, repeat the following:

new_value = 0.5 * (old_value + number / old_value)

difference = new_value - old_value

if difference is negative, change it to its positive value

old_value = new_value

Step 5: Continue the loop while difference is greater than or equal to TOLERANCE.

Step 6: Return new_value as the approximate square root. No function of math.h is used, so even the positive value of the difference is taken by hand instead of using fabs().

3. Test Case Table

In the tables below, the dimensions are in metres and the densities in kilograms per cubic metre. PI is taken as 3.1416 and GRAVITY as 9.8, as defined in `solid.h`.

Valid inputs

TC #	Menu	Input	Expected Output	Actual Output	Result
TC1	1	Sphere, radius 2	50.2656 m ²	50.2656 m ²	PASS
TC2	1	Cylinder, radius 2, height 5	87.9648 m ²	87.9648 m ²	PASS
TC3	1	Cone, radius 3, height 4 (slant height = 5)	75.3984 m ²	75.3984 m ²	PASS
TC4	1	Cuboid, 2 × 3 × 4	52.0000 m ²	52.0000 m ²	PASS
TC5	2	Sphere, radius 2	33.5104 m ³	33.5104 m ³	PASS
TC6	2	Cylinder, radius 2, height 5	62.8320 m ³	62.8320 m ³	PASS
TC7	2	Cone, radius 3, height 4	37.6992 m ³	37.6992 m ³	PASS
TC8	2	Cuboid, 2 × 3 × 4	24.0000 m ³	24.0000 m ³	PASS
TC9	3	Cylinder, radius 2	12.5664 m ²	12.5664 m ²	PASS
TC10	3	Cuboid, length 2, breadth 3	6.0000 m ²	6.0000 m ²	PASS
TC11	4	Density 1000, cuboid 2 × 3 × 4	24000.0000 kg	24000.0000 kg	PASS
TC12	5	Density 1000, cuboid 2 × 3 × 4	235200.0000 N	235200.0000 N	PASS
TC13	6	Elevation 10, density 1000, cuboid 2 × 3 × 4	2352000.0000 J	2352000.0000 J	PASS
TC14	7	Density 1000, cuboid 2 × 3 × 4	39200.0000 Pa	39200.0000 Pa	PASS
TC15	8	Fluid density 1000, cuboid 2 × 3 × 4	235200.0000 N	235200.0000 N	PASS
TC16	9	Solid 800, fluid 1000	The solid floats	The solid floats in the fluid.	PASS
TC17	9	Solid 1200, fluid 1000	The solid sinks	The solid sinks in the fluid.	PASS

Boundary inputs

TC #	Menu	Input	Expected Output	Actual Output	Result
TC18	9	Solid 1000, fluid 1000 (the two densities are equal)	The solid remains suspended	The solid remains suspended in the fluid.	PASS

TC #	Menu	Input	Expected Output	Actual Output	Result
TC19	6	Elevation 0 (the smallest allowed elevation)	0.0000 J	0.0000 J	PASS
TC20	1	Cone, radius 1, height 1 (slant height is not a whole number, it is $\sqrt{2}$)	7.5845 m ²	7.5845 m ²	PASS
TC21	2	Sphere, radius 0.001 (a very small dimension)	0.0000 m ³ (the real value is about 0.0000000042)	0.0000 m ³	PASS
TC22	3	Sphere selected	Message that the base area of a sphere is not defined, and return to the main menu	The base area of a sphere is not defined by the given dimensions... Returning to the main menu.	PASS

Invalid inputs

In every case below the program shows the error message and asks for the same value again. It does not stop and it does not take the wrong value.

TC #	Invalid input given	Expected Output	Actual Output	Result
TC23	Main menu choice = 99	Invalid choice message and the menu is shown again	Invalid choice! Enter 1 to 10.	PASS
TC24	Solid choice = 7, then 1	Error message, then ask again	Error: the choice must be from 1 to 4. Enter again:	PASS
TC25	Radius = -2, then 0, then 2 (invalid two times)	Error message both the times, then accept 2	The error message was shown 2 times, then the area came 50.2656	PASS
TC26	Elevation = -3, then 0	Error message, then ask again	Error: the elevation cannot be negative. Enter again:	PASS
TC27	Pressure, solid choice = 1 (a sphere is not allowed here), then 4	Error message, then ask again	Error: the choice must be from 2 to 4. Enter again:	PASS

4. Source Code

solid.h (48 lines)

```
/* solid.h
   Header guard, symbolic constants and the external declarations
   of all the functions defined in geometry.c, physics.c and
   math_utils.c */

#ifndef SOLID_H
#define SOLID_H

/* physical and mathematical constants */
#define PI      3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

/* the solid choices */
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

/* the buoyancy results */
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

/* functions defined in geometry.c */
extern double sphere_area(double radius);
extern double sphere_volume(double radius);
extern double cylinder_area(double radius, double height);
extern double cylinder_volume(double radius, double height);
extern double cone_area(double radius, double height);
extern double cone_volume(double radius, double height);
extern double cuboid_area(double length, double breadth, double height);
extern double cuboid_volume(double length, double breadth, double height);
extern double circular_base_area(double radius);
extern double cuboid_base_area(double length, double breadth);

/* functions defined in physics.c */
extern double calculate_mass(double density);
extern double calculate_weight(void);
extern double potential_energy(double elevation);
extern double calculate_pressure(double density);
extern double buoyant_force(double fluid_density);
extern int determine_buoyancy(double solid_density, double fluid_density);

/* function defined in math_utils.c */
extern double calculate_square_root(double number);

#endif
```

math_utils.c (38 lines)

```
/* math_utils.c
   Calculates the square root iteratively using the
   Newton-Raphson method. No function of math.h is used. */

#include "solid.h"

double calculate_square_root(double number)
{
    double old_value, new_value, difference;

    /* the real square root of a negative number does not exist */
    if (number < 0)
        return -1;

    if (number == 0)
        return 0;

    old_value = number;

    do {
        /* a better approximation */
        new_value = 0.5 * (old_value + number / old_value);

        /* how much the approximation changed */
```

```

        difference = new_value - old_value;

        /* take the positive value of the difference,
           because fabs() of math.h cannot be used */
        if (difference < 0)
            difference = -difference;

        /* use the new approximation in the next round */
        old_value = new_value;

    } while (difference >= TOLERANCE);

    return new_value;
}

```

geometry.c (68 lines)

```

/* geometry.c
   Calculates surface area, volume, base area and slant height */

#include "solid.h"

/* total surface area of a sphere */
double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}

/* volume of a sphere */
double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

/* total surface area of a cylinder */
double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

/* volume of a cylinder */
double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

/* total surface area of a cone */
double cone_area(double radius, double height)
{
    double slant_height;

    slant_height = calculate_square_root(radius * radius + height * height);

    return PI * radius * (radius + slant_height);
}

/* volume of a cone */
double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}

/* total surface area of a cuboid */
double cuboid_area(double length, double breadth, double height)
{
    return 2 * (length * breadth + breadth * height + length * height);
}

/* volume of a cuboid */
double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}

/* base area of a cylinder and a cone */
double circular_base_area(double radius)
{
    return PI * radius * radius;
}

/* base area of a cuboid */
double cuboid_base_area(double length, double breadth)
{

```

```

    return length * breadth;
}

```

physics.c (257 lines)

```

/* physics.c
Obtains the solid type and the required dimensions for the
physics operations and performs the corresponding
calculations */

#include <stdio.h>
#include "solid.h"

/* mass = density * volume */
double calculate_mass(double density)
{
    int solid;
    double radius, height, length, breadth, volume;

    volume = 0;

    printf("\n Select the solid:\n");
    printf("    1. Sphere    2. Cylinder    3. Cone    4. Cuboid\n");
    printf(" Enter your choice (1 to 4): ");
    scanf("%d", &solid);
    while (solid < SPHERE || solid > CUBOID) {
        printf(" Error: the choice must be from 1 to 4. Enter again: ");
        scanf("%d", &solid);
    }

    if (solid == SPHERE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        volume = sphere_volume(radius);
    }
    else if (solid == CYLINDER || solid == CONE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        if (solid == CYLINDER)
            volume = cylinder_volume(radius, height);
        else
            volume = cone_volume(radius, height);
    }
    else {
        printf(" Enter the length (m): ");
        scanf("%lf", &length);
        while (length <= 0) {
            printf(" Error: the length must be greater than 0. Enter again: ");
            scanf("%lf", &length);
        }
        printf(" Enter the breadth (m): ");
        scanf("%lf", &breadth);
        while (breadth <= 0) {
            printf(" Error: the breadth must be greater than 0. Enter again: ");
            scanf("%lf", &breadth);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        volume = cuboid_volume(length, breadth, height);
    }

    return density * volume;
}

/* weight = mass * GRAVITY */

```

```

double calculate_weight(void)
{
    double density, mass;

    printf(" Enter the density (kg/m^3): ");
    scanf("%lf", &density);
    while (density <= 0) {
        printf(" Error: the density must be greater than 0. Enter again: ");
        scanf("%lf", &density);
    }

    mass = calculate_mass(density);

    return mass * GRAVITY;
}

/* potential energy = mass * GRAVITY * elevation */
double potential_energy(double elevation)
{
    double density, mass;

    printf(" Enter the density (kg/m^3): ");
    scanf("%lf", &density);
    while (density <= 0) {
        printf(" Error: the density must be greater than 0. Enter again: ");
        scanf("%lf", &density);
    }

    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}

/* pressure = (density * volume * GRAVITY) / contact area.
The pressure of a sphere is not calculated, because its
ideal contact area is not defined by the given dimensions. */
double calculate_pressure(double density)
{
    int solid;
    double radius, height, length, breadth;
    double volume, contact_area;

    volume = 0;
    contact_area = 0;

    printf("\n Select the solid:\n");
    printf(" 2. Cylinder  3. Cone  4. Cuboid\n");
    printf(" Enter your choice (2 to 4): ");
    scanf("%d", &solid);
    while (solid < CYLINDER || solid > CUBOID) {
        printf(" Error: the choice must be from 2 to 4. Enter again: ");
        scanf("%d", &solid);
    }

    if (solid == CYLINDER || solid == CONE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        contact_area = circular_base_area(radius);
        if (solid == CYLINDER)
            volume = cylinder_volume(radius, height);
        else
            volume = cone_volume(radius, height);
    }
    else {
        printf(" Enter the length (m): ");
        scanf("%lf", &length);
        while (length <= 0) {
            printf(" Error: the length must be greater than 0. Enter again: ");
            scanf("%lf", &length);
        }
        printf(" Enter the breadth (m): ");
        scanf("%lf", &breadth);
        while (breadth <= 0) {

```

```

        printf(" Error: the breadth must be greater than 0. Enter again: ");
        scanf("%lf", &breadth);
    }
    printf(" Enter the height (m): ");
    scanf("%lf", &height);
    while (height <= 0) {
        printf(" Error: the height must be greater than 0. Enter again: ");
        scanf("%lf", &height);
    }
    contact_area = cuboid_base_area(length, breadth);
    volume = cuboid_volume(length, breadth, height);
}

return (density * volume * GRAVITY) / contact_area;
}

/* buoyant force = fluid density * volume * GRAVITY */
double buoyant_force(double fluid_density)
{
    int solid;
    double radius, height, length, breadth, volume;

    volume = 0;

    printf("\n Select the solid:\n");
    printf(" 1. Sphere  2. Cylinder  3. Cone  4. Cuboid\n");
    printf(" Enter your choice (1 to 4): ");
    scanf("%d", &solid);
    while (solid < SPHERE || solid > CUBOID) {
        printf(" Error: the choice must be from 1 to 4. Enter again: ");
        scanf("%d", &solid);
    }

    if (solid == SPHERE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        volume = sphere_volume(radius);
    }
    else if (solid == CYLINDER || solid == CONE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        if (solid == CYLINDER)
            volume = cylinder_volume(radius, height);
        else
            volume = cone_volume(radius, height);
    }
    else {
        printf(" Enter the length (m): ");
        scanf("%lf", &length);
        while (length <= 0) {
            printf(" Error: the length must be greater than 0. Enter again: ");
            scanf("%lf", &length);
        }
        printf(" Enter the breadth (m): ");
        scanf("%lf", &breadth);
        while (breadth <= 0) {
            printf(" Error: the breadth must be greater than 0. Enter again: ");
            scanf("%lf", &breadth);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        volume = cuboid_volume(length, breadth, height);
    }

    return fluid_density * volume * GRAVITY;
}

```

```

}

/* compare the two densities */
int determine_buoyancy(double solid_density, double fluid_density)
{
    if (solid_density > fluid_density)
        return SINKS;

    if (solid_density < fluid_density)
        return FLOATS;

    return SUSPENDED;
}

```

main.c (282 lines)

```

/* main.c
   Displays the main menu, obtains the common inputs, calls the
   required functions and displays the results */

#include <stdio.h>
#include "solid.h"

int main(void)
{
    int choice, solid, behaviour;
    double radius, height, length, breadth;
    double density, fluid_density, solid_density, elevation;
    double result;

    do {
        printf("\n===== MAIN MENU =====\n");
        printf(" 1. Surface area\n");
        printf(" 2. Volume\n");
        printf(" 3. Base area\n");
        printf(" 4. Mass\n");
        printf(" 5. Weight\n");
        printf(" 6. Gravitational potential energy\n");
        printf(" 7. Pressure on the base\n");
        printf(" 8. Buoyant force\n");
        printf(" 9. Buoyancy behaviour\n");
        printf("10. Exit\n");
        printf("===== \n");
        printf("Enter your choice (1 to 10): ");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("\n Select the solid:\n");
                printf("    1. Sphere    2. Cylinder    3. Cone    4. Cuboid\n");
                printf(" Enter your choice (1 to 4): ");
                scanf("%d", &solid);
                while (solid < SPHERE || solid > CUBOID) {
                    printf(" Error: the choice must be from 1 to 4. Enter again: ");
                    scanf("%d", &solid);
                }

                if (solid == SPHERE) {
                    printf(" Enter the radius (m): ");
                    scanf("%lf", &radius);
                    while (radius <= 0) {
                        printf(" Error: the radius must be greater than 0. Enter again: ");
                        scanf("%lf", &radius);
                    }
                    result = sphere_area(radius);
                }
            else if (solid == CYLINDER || solid == CONE) {
                printf(" Enter the radius (m): ");
                scanf("%lf", &radius);
                while (radius <= 0) {
                    printf(" Error: the radius must be greater than 0. Enter again: ");
                    scanf("%lf", &radius);
                }
                printf(" Enter the height (m): ");
                scanf("%lf", &height);
                while (height <= 0) {
                    printf(" Error: the height must be greater than 0. Enter again: ");
                    scanf("%lf", &height);
                }
                if (solid == CYLINDER)
                    result = cylinder_area(radius, height);
                else

```

```

        result = cone_area(radius, height);
    }
    else {
        printf(" Enter the length (m): ");
        scanf("%lf", &length);
        while (length <= 0) {
            printf(" Error: the length must be greater than 0. Enter again: ");
            scanf("%lf", &length);
        }
        printf(" Enter the breadth (m): ");
        scanf("%lf", &breadth);
        while (breadth <= 0) {
            printf(" Error: the breadth must be greater than 0. Enter again: ");
            scanf("%lf", &breadth);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        result = cuboid_area(length, breadth, height);
    }
    printf("\n Total surface area : %.4f square metres\n", result);
    break;
}

case 2:
    printf("\n Select the solid:\n");
    printf(" 1. Sphere 2. Cylinder 3. Cone 4. Cuboid\n");
    printf(" Enter your choice (1 to 4): ");
    scanf("%d", &solid);
    while (solid < SPHERE || solid > CUBOID) {
        printf(" Error: the choice must be from 1 to 4. Enter again: ");
        scanf("%d", &solid);
    }

    if (solid == SPHERE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        result = sphere_volume(radius);
    }
    else if (solid == CYLINDER || solid == CONE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        if (solid == CYLINDER)
            result = cylinder_volume(radius, height);
        else
            result = cone_volume(radius, height);
    }
    else {
        printf(" Enter the length (m): ");
        scanf("%lf", &length);
        while (length <= 0) {
            printf(" Error: the length must be greater than 0. Enter again: ");
            scanf("%lf", &length);
        }
        printf(" Enter the breadth (m): ");
        scanf("%lf", &breadth);
        while (breadth <= 0) {
            printf(" Error: the breadth must be greater than 0. Enter again: ");
            scanf("%lf", &breadth);
        }
        printf(" Enter the height (m): ");
        scanf("%lf", &height);
        while (height <= 0) {
            printf(" Error: the height must be greater than 0. Enter again: ");
            scanf("%lf", &height);
        }
        result = cuboid_volume(length, breadth, height);
    }

```

```

    }
    printf("\n Volume : %.4f cubic metres\n", result);
    break;

case 3:
    printf("\n Select the solid:\n");
    printf("    1. Sphere    2. Cylinder    3. Cone    4. Cuboid\n");
    printf(" Enter your choice (1 to 4): ");
    scanf("%d", &solid);
    while (solid < SPHERE || solid > CUBOID) {
        printf(" Error: the choice must be from 1 to 4. Enter again: ");
        scanf("%d", &solid);
    }

    if (solid == SPHERE) {
        printf("\n The base area of a sphere is not defined by the given\n");
        printf(" dimensions, so it cannot be calculated.\n");
        printf(" Returning to the main menu.\n");
        break;
    }

    if (solid == CYLINDER || solid == CONE) {
        printf(" Enter the radius (m): ");
        scanf("%lf", &radius);
        while (radius <= 0) {
            printf(" Error: the radius must be greater than 0. Enter again: ");
            scanf("%lf", &radius);
        }
        result = circular_base_area(radius);
    }
    else {
        printf(" Enter the length (m): ");
        scanf("%lf", &length);
        while (length <= 0) {
            printf(" Error: the length must be greater than 0. Enter again: ");
            scanf("%lf", &length);
        }
        printf(" Enter the breadth (m): ");
        scanf("%lf", &breadth);
        while (breadth <= 0) {
            printf(" Error: the breadth must be greater than 0. Enter again: ");
            scanf("%lf", &breadth);
        }
        result = cuboid_base_area(length, breadth);
    }
    printf("\n Base area : %.4f square metres\n", result);
    break;

case 4:
    printf(" Enter the density (kg/m^3): ");
    scanf("%lf", &density);
    while (density <= 0) {
        printf(" Error: the density must be greater than 0. Enter again: ");
        scanf("%lf", &density);
    }
    result = calculate_mass(density);
    printf("\n Mass : %.4f kilograms\n", result);
    break;

case 5:
    result = calculate_weight();
    printf("\n Weight : %.4f newtons\n", result);
    break;

case 6:
    printf(" Enter the elevation (m): ");
    scanf("%lf", &elevation);
    while (elevation < 0) {
        printf(" Error: the elevation cannot be negative. Enter again: ");
        scanf("%lf", &elevation);
    }
    result = potential_energy(elevation);
    printf("\n Gravitational potential energy : %.4f joules\n", result);
    break;

case 7:
    printf(" Enter the density (kg/m^3): ");
    scanf("%lf", &density);
    while (density <= 0) {
        printf(" Error: the density must be greater than 0. Enter again: ");
        scanf("%lf", &density);
    }
    result = calculate_pressure(density);

```

```

        printf("\n Pressure on the base : %.4f pascals\n", result);
        break;

    case 8:
        printf(" Enter the fluid density (kg/m^3): ");
        scanf("%lf", &fluid_density);
        while (fluid_density <= 0) {
            printf(" Error: the density must be greater than 0. Enter again: ");
            scanf("%lf", &fluid_density);
        }
        result = buoyant_force(fluid_density);
        printf("\n Buoyant force : %.4f newtons\n", result);
        break;

    case 9:
        printf(" Enter the density of the solid (kg/m^3): ");
        scanf("%lf", &solid_density);
        while (solid_density <= 0) {
            printf(" Error: the density must be greater than 0. Enter again: ");
            scanf("%lf", &solid_density);
        }
        printf(" Enter the density of the fluid (kg/m^3): ");
        scanf("%lf", &fluid_density);
        while (fluid_density <= 0) {
            printf(" Error: the density must be greater than 0. Enter again: ");
            scanf("%lf", &fluid_density);
        }
        behaviour = determine_buoyancy(solid_density, fluid_density);
        if (behaviour == SINKS)
            printf("\n The solid sinks in the fluid.\n");
        else if (behaviour == FLOATS)
            printf("\n The solid floats in the fluid.\n");
        else
            printf("\n The solid remains suspended in the fluid.\n");
        break;

    case 10:
        printf("\nExiting the program.\n");
        break;

    default:
        printf("\nInvalid choice! Enter 1 to 10.\n");
        break;
}

} while (choice != 10);

return 0;
}

```

5. Compilation and Output

All the five files were written in the `SolidProject` folder using the `nano` editor. Since the project has more than one source file, all the `.c` files are given together in one gcc command. The header file is not written in the command, because it is already included inside the source files.

```
$ cd ~/Desktop/MyAssignment/SolidProject
$ nano solid.h
$ nano math_utils.c
$ nano geometry.c
$ nano physics.c
$ nano main.c
$ gcc main.c geometry.c physics.c math_utils.c -o solid_project
$ ./solid_project
```

The project compiled in the first attempt itself, without any error or warning.

Surface area and volume of the four solids

```
SolidProject — zsh — 132x38
adilshakil@ADILs-MacBook-Air SolidProject % ./solid_project

===== MAIN MENU =====
1. Surface area      2. Volume      3. Base area
4. Mass              5. Weight      6. Gravitational potential energy
7. Pressure on the base  8. Buoyant force
9. Buoyancy behaviour  10. Exit
=====
Enter your choice (1 to 10): 1

Select the solid:
1. Sphere    2. Cylinder    3. Cone    4. Cuboid
Enter your choice (1 to 4): 1
Enter the radius (m): 2
Total surface area : 50.2656 square metres

Enter your choice (1 to 10): 1
Enter your choice (1 to 4): 2    radius 2, height 5
Total surface area : 87.9648 square metres

Enter your choice (1 to 10): 1
Enter your choice (1 to 4): 3    radius 3, height 4
Total surface area : 75.3984 square metres

Enter your choice (1 to 10): 1
Enter your choice (1 to 4): 4    length 2, breadth 3, height 4
Total surface area : 52.0000 square metres

Enter your choice (1 to 10): 2
Enter your choice (1 to 4): 1    radius 2
Volume : 33.5104 cubic metres

Enter your choice (1 to 10): 2
Enter your choice (1 to 4): 2    radius 2, height 5
Volume : 62.8320 cubic metres

Enter your choice (1 to 10): 2
Enter your choice (1 to 4): 3    radius 3, height 4
Volume : 37.6992 cubic metres

Enter your choice (1 to 10): 2
Enter your choice (1 to 4): 4    length 2, breadth 3, height 4
Volume : 24.0000 cubic metres

Enter your choice (1 to 10): 10
Exiting the program.
adilshakil@ADILs-MacBook-Air SolidProject %
```

The main menu and the solid submenu are displayed again before every choice. They are shown fully only the first time, and after that only the entered values are shown, so that the output stays readable.

Base area, and the sphere which is not allowed here



```
adilshakil@ADILs-MacBook-Air SolidProject % ./solid_project
Enter your choice (1 to 10): 3
Enter your choice (1 to 4): 2
Enter the radius (m): 2
Base area : 12.5664 square metres

Enter your choice (1 to 10): 3
Enter your choice (1 to 4): 4
Enter the length (m): 2
Enter the breadth (m): 3
Base area : 6.0000 square metres

Enter your choice (1 to 10): 3
Enter your choice (1 to 4): 1

The base area of a sphere is not defined by the given
dimensions, so it cannot be calculated.
Returning to the main menu.

Enter your choice (1 to 10): 10
Exiting the program.
adilshakil@ADILs-MacBook-Air SolidProject %
```

Mass, weight, potential energy, pressure, buoyant force and buoyancy



```
adilshakil@ADILs-MacBook-Air SolidProject % ./solid_project
Enter your choice (1 to 10): 4
  Enter the density (kg/m^3): 1000
  Enter your choice (1 to 4): 4    length 2, breadth 3, height 4
  Mass : 24000.0000 kilograms

Enter your choice (1 to 10): 5
  Enter the density (kg/m^3): 1000
  Enter your choice (1 to 4): 4    length 2, breadth 3, height 4
  Weight : 235200.0000 newtons

Enter your choice (1 to 10): 6
  Enter the elevation (m): 10
  Enter the density (kg/m^3): 1000
  Enter your choice (1 to 4): 4    length 2, breadth 3, height 4
  Gravitational potential energy : 2352000.0000 joules

Enter your choice (1 to 10): 7
  Enter the density (kg/m^3): 1000

  Select the solid:
    2. Cylinder    3. Cone    4. Cuboid
  Enter your choice (2 to 4): 4    length 2, breadth 3, height 4
  Pressure on the base : 39200.0000 pascals

Enter your choice (1 to 10): 8
  Enter the fluid density (kg/m^3): 1000
  Enter your choice (1 to 4): 4    length 2, breadth 3, height 4
  Buoyant force : 235200.0000 newtons

Enter your choice (1 to 10): 9
  Enter the density of the solid (kg/m^3): 800
  Enter the density of the fluid (kg/m^3): 1000
  The solid floats in the fluid.

Enter your choice (1 to 10): 9    solid 1200, fluid 1000
  The solid sinks in the fluid.

Enter your choice (1 to 10): 9    solid 1000, fluid 1000
  The solid remains suspended in the fluid.

Enter your choice (1 to 10): 10
Exiting the program.
adilshakil@ADILs-MacBook-Air SolidProject %
```

Boundary cases and invalid inputs

```
adilshakil@ADILs-MacBook-Air SolidProject % ./solid_project
Enter your choice (1 to 10): 1
  Enter your choice (1 to 4): 3
  Enter the radius (m): 1
  Enter the height (m): 1
  Total surface area : 7.5845 square metres          (slant height is the square root of 2)

Enter your choice (1 to 10): 2
  Enter your choice (1 to 4): 1
  Enter the radius (m): 0.001
  Volume : 0.0000 cubic metres                      (the real value is about 0.0000000042)

Enter your choice (1 to 10): 99

Invalid choice! Enter 1 to 10.

Enter your choice (1 to 10): 1
  Enter your choice (1 to 4): 7
  Error: the choice must be from 1 to 4. Enter again: 1
  Enter the radius (m): -2
  Error: the radius must be greater than 0. Enter again: 0
  Error: the radius must be greater than 0. Enter again: 2
  Total surface area : 50.2656 square metres

Enter your choice (1 to 10): 6
  Enter the elevation (m): -3
  Error: the elevation cannot be negative. Enter again: 0
  Enter the density (kg/m^3): 1000
  Enter your choice (1 to 4): 4    length 2, breadth 3, height 4
  Gravitational potential energy : 0.0000 joules

Enter your choice (1 to 10): 7
  Enter the density (kg/m^3): 1000
  Enter your choice (2 to 4): 1
  Error: the choice must be from 2 to 4. Enter again: 4    length 2, breadth 3, height 4
  Pressure on the base : 39200.0000 pascals

Enter your choice (1 to 10): 10
Exiting the program.
adilshakil@ADILs-MacBook-Air SolidProject %
```